Experiment 2 : Uber Fare Prediction with Linear, Rid and Lasso Regression

Aim: Predict the price of an Uber ride from trip features and compare Linear, Ridge and Lasso regression.

In [1]:
```
# A52 Dipanshu Ambilkar
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score, mean_squared_error
```

In [2]:
```
rng = np.random.default_rng(7)
n = 500
distance = rng.uniform(1, 30, n)
print("First 5 distances:", distance[:5].round(1))
```

```
First 5 distances: [19.1 27.  23.5  7.5  9.7]
```

In [3]:
```
duration = distance * rng.uniform(1.5, 3, n)
passengers = rng.integers(1, 5, n)
fare = 30 + 12 * distance + 1.5 * duration + rng.normal(0, 15, n)
print("First 5 fares:", fare[:5].round(2))
```

```
First 5 fares: [324.06 442.56 418.36 164.25 179.66]
```

In [4]:
```
df = pd.DataFrame({"distance": distance, "duration": duration,
                   "passengers": passengers, "fare": fare})
print(df.head())
```

```
    distance   duration  passengers        fare
0  19.127769  40.749108           3  324.056778
1  27.019200  74.088630           4  442.560731
2  23.494885  64.408146           2  418.355322
3   7.531009  16.647355           4  164.248757
4   9.704822  25.735889           1  179.663582
```

In [5]:
```
print("Shape before cleaning:", df.shape)
```

```
Shape before cleaning: (500, 4)
```

In [6]:
```
df = df.dropna()
print("Rows after cleaning:", len(df))
```

```
Rows after cleaning: 500
```

In [7]:
```
q1, q3 = df["fare"].quantile([0.25, 0.75])
iqr = q3 - q1
print("Q1:", round(q1, 2), "| Q3:", round(q3, 2), "| IQR:", round(iqr, 2)
```

```
Q1: 160.23 | Q3: 381.66 | IQR: 221.43
```

In [8]:
```
outliers = df[(df["fare"] < q1 - 1.5 * iqr) | (df["fare"] > q3 + 1.5 * iq
print("Outliers found:", len(outliers))
df = df.drop(outliers.index)
print("Rows after removing outliers:", len(df))
```

```
Outliers found: 0
Rows after removing outliers: 500
```

In [9]:
```
print("Correlation with fare:")
print(df.corr()["fare"].round(3))
```

```
Correlation with fare:
distance      0.988
duration      0.946
passengers   -0.033
fare          1.000
Name: fare, dtype: float64
```

In [10]:
```
X = df[["distance", "duration", "passengers"]]
y = df["fare"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=7)
print("Train size:", len(X_train), "| Test size:", len(X_test))
```

```
Train size: 400 | Test size: 100
```

In [11]:
```
model = LinearRegression()
model.fit(X_train, y_train)
pred = model.predict(X_test)
print(f"Linear   R2 = {r2_score(y_test, pred):.4f}   RMSE = {mean_squared
```

```
Linear   R2 = 0.9861   RMSE = 14.03
```

In [12]:
```
model = Ridge()
model.fit(X_train, y_train)
pred = model.predict(X_test)
print(f"Ridge    R2 = {r2_score(y_test, pred):.4f}   RMSE = {mean_squared
```

```
Ridge    R2 = 0.9861   RMSE = 14.03
```

In [13]:
```
model = Lasso()
model.fit(X_train, y_train)
pred = model.predict(X_test)
print(f"Lasso    R2 = {r2_score(y_test, pred):.4f}   RMSE = {mean_squared
```

```
Lasso    R2 = 0.9861   RMSE = 14.06
```